Swytchcode PPI next steps

Purpose

This short steps helps us understand how you explore a new developer tool, solve problems, communicate your work, and share useful product feedback. It also helps us identify friction in Swytchcode's first-time onboarding. We do not expect you to be an experienced developer or to complete everything perfectly.

Time and deadline

Please submit your work within 72 hours of receiving this assignment. Expected effort: approximately 2–3 hours. Submission date: **5th November 2026 (11:59 pm)**.

What to do

- 1. Explore Swytchcode using the CLI. Start at https://swytchcode.com and find the current CLI installation instructions: https://docs.swytchcode.com. Install the CLI and initialize a project.
- 2. Inspect the Wrekenfile generated during project initialization. You do not need to create one from scratch. Explore the available integrations, methods and workflows.
- 3. Try to execute at least one API operation through Swytchcode. You may use an existing integration or the generated project files. You may modify or extend the Wrekenfile if useful, but this is optional.
- 4. Build a small, practical example around the API operation. A complete AI agent is not required. A focused working example or a clearly documented attempt if you hit a blocker, is enough.
- 5. Explain Swytchcode in your own words (approximately 150–250 words): what it does, what problem it solves, how a developer uses it, and one practical example.
- 6. Create one marketing asset: a Reddit post, LinkedIn post, 2–3 minute demo video, or demo script. You do not need to publish it publicly.

Use of AI and other resources

You may use ChatGPT, Claude, Gemini, Copilot, documentation, tutorials, GitHub and other resources. AI use is allowed and encouraged. Please make sure you understand the work you submit and can explain the decisions and commands. Briefly disclose which AI tools you used and how.

Never commit API keys, passwords, access tokens or other secrets. Redact secrets from screenshots and logs.

Keep a friction diary

As you work, note confusing or unsuccessful steps while they are fresh. For each friction point, record: the step you were attempting; the command or action; what you expected; what actually

happened (including the exact error); what you tried next (documentation, search, AI prompt, etc.); and whether you resolved it.

Please include the exact documentation page, heading, command or search phrase you consulted where relevant. If you used AI, include the prompt and whether the answer worked. Honest, specific feedback is more useful than a general rating.

What to submit

A GitHub repository (public, or private with access shared) containing a README, relevant project files including the Wrekenfile, the commands needed to reproduce your attempt, what worked or did not work, and any useful screenshots or logs with secrets removed.

A demo video of up to 3 minutes showing your project, CLI commands, the API execution (or the blocker you encountered), and one thing you learned. Loom, Google Drive or an unlisted YouTube link is fine. Professional editing is not expected.

The Google Form: https://docs.google.com/forms/d/e/1FAIpQLSeBHYcRhWEuffCSoJJC5NiuToutR_ElQE4jvPDjueafRVG8gg/viewform. Include your onboarding and friction diary answers, your explanation of Swytchcode, your marketing asset, repository and demo links, and your AI usage.

If you need support

If you get stuck and need help, create a ticket in this Discord channel: https://discord.com/channels/1295559054071959592/1515632131471048866 and tag **@chilarai**. Include the command or action, the error, and what you have already tried. Do not share credentials or secrets.

How selection will work

We will review the submitted answers and project work, then invite a small number of students for an interview before making the final selection. Only a few candidates will be invited to interview, and submitting the task does not guarantee an interview or selection.

We are interested in how you learn, approach problems, explain your work and identify product friction, not just whether everything works. A small, clearly explained attempt is better than an ambitious unfinished project. If you encounter a blocker, document it rather than hiding it.

Suggested README structure

Problem and intended user; API/integration explored; setup and commands; what worked; what did not work and errors encountered; how to reproduce; link to demo; notes on the generated Wrekenfile.